

A Dynamic Data Structure for 3-d Convex Hulls and 2-d Nearest Neighbor Queries

Задача

Требуется поддерживать удаления и вставки на множестве из точек в пространстве, а также находить самую удаленную точку в заданном направлении.

Связь с ближайшими соседями

Рассмотрим также задачу поиска ближайшего соседа на плоскости: требуется поддерживать вставки и удаления на множестве точек на плоскости и уметь находить ближайшую точку из множества к произвольной точке плоскости. Понятно, что эта задача сводится к поставленной выше при помощи проекции на параболоид.

Полезные определения

Для поиска экстремальной точки в данном направлении достаточно хранить выпуклую оболочку. Достаточно рассматривать верхнюю оболочку, ведь для нижней задача аналогична. Также будем решать не оригинальную версию задачи, а двойственную: требуется поддерживать множество H из n плоскостей так что для любой вертикальной прямой q мы можем находить самую нижнюю плоскость, назовем ее $ANS(H, q)$. Переход от обычной задачи к двойственной осуществляется двойственным преобразованием относительно параболоида.

Определение 1. Пусть есть множество H из n плоскостей и какой-то регион пространства γ . Тогда $(1/r)$ -разрезание γ — это набор клеток (тетраэдров), такой что объединение замыканий всех клеток содержит γ , и каждый тетраэдр пересекает не более $\lceil n/r \rceil$ плоскостей из H .

Определение 2. Списком конфликтов клетки Δ назовем подмножество плоскостей H , пересекающих Δ . Обозначим его как H_Δ .

Определение 3. $\leq k$ -уровнем называется замыкание множества всех точек q таких что не более k плоскостей H находятся ниже q .

Лемма 1. Для любого множества H из n плоскостей и параметра r существует $O(1/r)$ -разрезание $\leq n/r$ -уровня H на $O(r)$ клеток. Такое разрезание, вместе со списками конфликтов, строится за ожидаемое время $O(n \log n)$. Клетки — это неограниченные вертикальные тетраэдры (такие, у которых одна из вершин имеет координаты $(0, 0, -\infty)$).

Эта лемма — применение Shallow Cutting леммы Матушка, где сэмплированием набиралось нужное разрезание.

Теперь мы сначала построим вспомогательную структуру данных, которая будет поддерживать только удаления, а затем с ее помощью построим итоговую структуру хранения плоскостей.

Вспомогательная структура

Пусть у нас изначально есть множество S из n плоскостей. Мы будем также иметь множество $S_{live} \subset S$ "живых" плоскостей. Изначально гарантируется что S_{live} содержит хотя бы половину плоскостей из S . В процессе мы будем убивать какие-то плоскости из S_{live} . Статическое множество S_{static} будет нужно для ответа на запросы.

Построение этой структуры состоит из логарифмического количества слоев, где на каждом слое мы строим разрезание и удаляем какие-то "плохие" плоскости, которые создают слишком много конфликтов. Псевдокод построения S_{live} и S_{static} :

Algorithm 1 Construct(S)

```

1:  $S_0 = S$ 
2: for  $i = 1, \dots, \lceil \log n \rceil$  do
3:    $T_i = (1/2^i)$ -разрезание ( $\leq \lceil |S_{i-1}|/c2^i \rceil$ )-уровня  $S_{i-1}$  на  $c'2^i$  клеток
4:    $S_i = S_{i-1} - \{\text{все плоскости пересекающие более } 4c' \lceil \log n \rceil \text{ клеток } T_i\}$ 
5:   for всех  $\Delta \in T_i$  do
6:      $\lfloor$    Посчитать список конфликтов  $(S_i)_\Delta$  и определить  $j_\Delta = 0$ 
7:    $S_{live} = S_{static} = S_{\lceil \log n \rceil}$ 

```

После каждого удаления плоскости мы будем пересчитывать списки конфликтов в которых участвовала данная плоскость. Когда для конкретного списка конфликтов будет удалена какая-то его часть, мы будем убивать (удалять из S_{live} все оставшиеся в списке плоскости. Зачем так делать? Об этом далее.

Давайте пока зафиксируем константу $c'' > 4c$ и посмотрим на код для удаления плоскости из S .

Algorithm 2 Delete(S, h)

```

1: Удаляем  $h$  из  $S$ , а также удаляем ее из  $S_{live}$  (если она там есть)
2: for всех  $i$  и всех  $\Delta \in T_i$ , для которых  $h \in (S_i)_\Delta$  do
3:    $j_\Delta + = 1$ 
4:   if  $j_\Delta = \lceil |(S_i)_\Delta|/c'' \rceil$  then
5:      $\lfloor$    удалить все плоскости  $(S_i)_\Delta$  из  $S_{live}$ 

```

Сейчас мы что-то поймем про то как эта структура работает

Лемма 2. *a) $construct()$ занимает $O(n \log^2 n)$ ожидаемого времени и в итоге в S_{live} оказывается хотя бы $n/2$ плоскостей*

b) n_D выполнений $delete()$ занимают $O(n_D \log^2 n)$ времени и удаляют хотя бы $O(n_D \log^2 n)$ плоскостей из S_{live}

c) (для чего это все затевалось) Для любой вертикальной прямой q , если $ANS(S, q) \in S_{live}$, то $ANS(S, q) = ANS(S_{static}, q)$

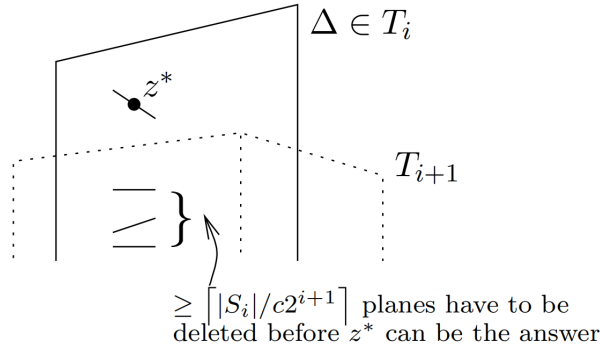


Рис. 1: Доказательство Леммы 2 пункта c: $\Delta \in T_i$ — тетраэдр повыше, T_{i+1} — слои пониже. Для того, чтобы z^* оказалось нижней в данной точке надо удалить много плоскостей.

Доказательство. *a)* Число пар (клетка, плоскость ее пересекающая) в $T_i \times S_i$ не больше чем $c'2^i \cdot \lceil n/2^i \rceil \leq 2c'n$ по построению. Тогда 4-я строчка алгоритма $construct()$ удаляет за раз не более чем $n/(2\lceil \log n \rceil)$ плоскостей. Тогда суммарно мы уберем не более чем $n/2$ плоскостей. Ограничение по времени получается из Леммы 1: мы применяем ее $\log n$ раз.

b) Для каждого i $h \in S_i$ пересекает не более чем $4c'\lceil \log n \rceil$ клеток $\Delta \in T_i$. Тогда количество увеличений j -шек выполненное каждым $delete$ -ом равно $O(\log^2 n)$. Пусть каждое увеличение стоит c'' единиц. Поскольку нужно накопить $|(S_i)_\Delta|$ единиц прежде чем выполняется $\leq |(S_i)_\Delta|$ удалений на строчке 4, то общее число удалений ограничено суммой единиц, а она равна $O(n_D \log^2 n)$. Отсюда же следует ограничение по времени (если все необходимые указатели уже расставили).

c) Пусть $h^* = ANS(S, q) \in S_{live}$, а также $z^* = q \cap h^*$. Если z^* лежит внутри какого-то тетраэдра $\Delta \in T_{\lceil \log n \rceil}$, то поскольку $|(S_{static})_\Delta| \leq 1$, $h^* = ANS(S_{static}, q)$, что и требовалось.

Тогда z^* лежит внутри тетраэдра $\Delta \in T_i$, но вне всех тетраэдров T_{i+1} для какого-то i . Отметим, что $S_{live} \subset S_{static} \subset S_i$ для любого i . Поскольку T_{i+1} содержит ($\leq \lceil |S_i|/c2^{i+1} \rceil$)-уровень S_i , то количество плоскостей ниже z^* хотя бы $\lceil |S_i|/c2^{i+1} \rceil \geq \lceil n/c2^{i+2} \rceil \geq \lceil |(S_i)_\Delta|/4c \rceil$. Чтобы z^* оказалась ответом, все эти плоскости должны были быть удалены из S . Но тогда из строчки 4 $delete()$ все плоскости $(S_i)_\Delta$, включая h^* , должны были быть удалены из S_{live} , противоречие. То есть этот случай вообще невозможен. $\square$

Основная структура данных

Теперь на основе предыдущего куска построим структуру, которая поддерживает удаления и вставки. Будем хранить $O(\log n)$ подмножеств S , каждое из которых является структурой из предыдущей секции. Будем поддерживать следующее свойство: в любой момент времени все полученные S_{live} не пересекаются и $\bigcup_S S_{live} = H$. Однако сами подмножества S не обязательно не пересекаются (в них могут быть мертвые плоскости). Для каждого множества S мы храним нижнюю оболочку S_{static} (с ее помощью мы будем отвечать на запросы).

Построение такой структуры мы очевидно реализуем через логарифмический вызов `construct()`:

Algorithm 3 preprocess(S)

- 1: **if** $S \neq \emptyset$ **then**
 - 2: Добавляем множество S , вызываем `construct(S)`, а затем `preprocess(S-Slive)`
-

Чтобы вставить новую плоскость в структуру мы просто добавляем в коллекцию новое множества размера 1. Чтобы сохранять логарифмический размер структуры, мы потом будем мерджить множества одинакового размера (а также окончательно хоронить мертвые плоскости по пути). Для множества S определим его глубину как $\lfloor \log |S_{live}| \rfloor$. Пусть $b = 16$.

Algorithm 4 insert(h)

- 1: добавляем новое множество $S = \{h\}$ где $S_{live} = S_{static} = S$.
 - 2: **while** существует b множеств $S^{(1)}, \dots, S^{(b)}$ на одной глубине **do**
 - 3: удалить множества $S^{(1)}, \dots, S^{(b)}$ и вызвать `preprocess( $S_{live}^{(1)} \cup \dots \cup S_{live}^{(b)}$ )`
-

После уборки лишних множеств на каждой глубине останется не более чем $b - 1$ множество, что даст структуре логарифмический размер.

Чтобы удалить плоскость h из H будем вызывать `delete` из прошлой секции для всех S содержащих h . При этом мы можем убить какие-то плоскости по дороге. Идея в том чтобы заново вставить эти плоскости в структуру новым множеством.

Algorithm 5 delete(h)

- 1: $A = \emptyset$
 - 2: **for** всех множеств S , содержащих h **do**
 - 3: вызывать `delete(S, h)` и все новоубитые плоскости добавить в A .
 - 4: **for** всех $g \in A - h$ **do**
 - 5: вызывать `insert(g)`
-

Отвечать на запросы будем совсем легко: просто пройдемся по всем S_{static} .

Algorithm 6 query(h)

- 1: $B = \emptyset$
 - 2: **for** всех множеств S , для которых $ANS(S_{static}, h) \in S_{live}$ **do**
 - 3: добавить $ANS(S_{static}, h)$ в B
 - 4: **return** $ANS(B, q)$
-

На этом определение структуры наконец закончилось. Осталось только доказать что это работает за крутое амортизированное время.

Теорема 1. *a) Любая последовательность из n_I `insert()` и n_D `delete()` на множестве с изначальным размером n_0 работает за следующее ожидаемое амортизированное время $O(n_0 \log^2 n + n_I \log^3 n + n_D \log^6 n)$, где n — максимальный размер множества на протяжении работы алгоритма.*

b) `query()` отвечает на вопросы верно за время $O(\log^2 n)$.

Доказательство. **a)** Определим потенциал структуры данных как $P = \sum_{S_{live}} |S_{live}| \log |S_{live}|$.

В любой момент времени потенциал ограничен сверху величиной $n \log n$ вследствие непересекаемости подмножеств S_{live} .

Стоимость предобработки набора из n_0 плоскостей, согласно лемме 2, составляет

$$O\left(\sum_{j=0}^{\infty} \binom{n_0}{2^j} \log^2 \binom{n_0}{2^j}\right) = O(n_0 \log^2 n_0).$$

Посмотрим на `insert`: рассмотрим одну итерацию строк 2–3. Пусть множества $S^{(i)}$ находятся на глубине k . Стоимость этого шага равна $O(p \log^2 p)$, где $p = |S_{live}^{(1)}| + \dots + |S_{live}^{(b)}|$. Заметим, что $p \geq 2^{k+4}$ при нашем

выборе $b = 16$. Увеличение потенциала на этом шаге составляет не менее

$$\sum_{j=1}^{\infty} 2^{\frac{p}{j}} \log \frac{p}{2^j} - \sum_{i=1}^b |S_{\text{live}}^{(i)}| \log |S_{\text{live}}^{(i)}| \geq p \log p \sum_{j=1}^{\infty} \frac{1}{2^j} - p \sum_{j=1}^{\infty} \frac{j}{2^j} - p(k+1) \geq p(k+4) - 2p - p(k+1) = p.$$

Таким образом, мы можем ограничить стоимость вставки увеличением потенциала, умноженным на множитель $\log^2 n$.

Теперь посмотрим на delete: рассмотрим строки 1–3. По лемме 2 размер всех порождаемых множеств A составляет не более $O(\log^2 n)$, умноженного на количество вызовов $\text{delete}(S, h)$. Поскольку это количество равно $O(n_D \log n)$ для всей последовательности обновлений, суммарный размер всех A , а также стоимость вызовов $\text{delete}(S, h)$, составляет $O(n_D \log^3 n)$. Однако потенциал при этом уменьшается. Докажем случайное неравенство:

$$(x + y) \log(x + y) - x \log x = y \log(x + y) + x \log(1 + y/x) \leq y \log(x + y) + x \cdot O(y/x) = y \log(x + y) + O(y),$$

получаем, что уменьшение потенциала на этом шаге не превосходит

$$|S_{\text{live}}| \log |S_{\text{live}}| - (|S_{\text{live}}| - |A|) \log(|S_{\text{live}}| - |A|) \leq O(|A| \log |S_{\text{live}}|) \leq O(|A| \log n).$$

Для всей последовательности обновлений суммарное уменьшение потенциала, таким образом, составляет $O(n_D \log^4 n)$. Время работы строки 4 ограничено увеличением потенциала, умноженным на $\log^2 n$, как уже показано выше.

Следовательно, общая стоимость всей последовательности обновлений ограничена величиной

$$O(n_D \log^3 n + (\text{суммарное увеличение потенциала}) \log^2 n),$$

где суммарное увеличение потенциала = (конечный потенциал – начальный потенциал) + суммарное уменьшение потенциала = $O(n \log n - n_0 \log n_0 + n_D \log^4 n) = O(n_I \log n + n_D \log^4 n)$.

Таким образом, мы получаем искомую оценку времени.

б) Докажем корректность: по построению $ANS(H, q)$ принадлежит S_{live} для некоторого подмножества S , и по лемме 2 $ANS(H, q) = ANS(S, q) = ANS(S_{\text{static}}, q)$. Время работы следует из того, что для каждого из $O(\log n)$ подмножеств S вычисление $ANS(S, q)$ занимает $O(\log n)$ времени благодаря какому-то известному статистическому результату, который мы примем как черный ящик. □

Из интересного: можно довольно легко адаптировать алгоритм, чтобы отвечать на вопросы про k нижних плоскостей с небольшим ухудшением по времени (тоже полилогарифм, только большей степени).